

How do you run CS223 labs on a Mac without Vivado?

This is a step by step setup for a Mac with Apple Silicon. I ran all of it on an M2 with 8 GB of RAM, macOS 15 and a Digilent Basys3 board, on 19 September 2026.

You do not need to know what compiling is or how FPGA tools work. If you can copy a command, paste it into the terminal and press Enter, you can finish this. Every step tells you what you are doing, why you need it and what you should see when it worked.

The compiling takes roughly 20 minutes in total while you wait. The rest is downloads and copy and paste.

I only tested this on Apple Silicon, which means M1 and later. I have not tried it on an Intel Mac.

You can download the five project files. Section 8 lists them in full, but you don't have to copy them out of this PDF. The download link is in section 8, and it gives you all five files with their tabs and indentation intact.

If a step fails. Do not skip ahead, because every step needs the one before it. Read the last lines in the terminal. Most errors I hit are listed in step 16 with the fix. If yours is not there, send me the last ten lines of the terminal on LinkedIn and tell me which step you were on.

Contents

1	What are you building?	2
2	What do you need before you start?	2
3	Which tools come ready-made?	3
4	Why do you need a Python environment?	3
5	How do you build nextpnr-xilinx?	3
6	How do you make the chip database?	4
7	How do you build the prjxray tools?	5
8	What goes in your project folder?	5
9	How do you run it?	9
10	How do you use it for your own lab?	9
11	How do you set up VS Code?	10
12	Which course file breaks?	10
13	What was tested?	11
14	What can it actually not do?	11
15	What should you be concerned about?	12
16	What went wrong during my setup?	12
17	Which versions did I use?	12
18	What is not verified?	13

1. What are you building?

Vivado does five jobs for a CS223 lab. You are going to install one free tool for each job and connect them with a small file, so that one command runs the whole chain.

```
your_design.sv
|
|-> Icarus Verilog    -> simulation
|-> Yosys            -> synthesis
|-> nextpnr-xilinx   -> place and route
|-> fasm2frames      -> configuration frames
|-> xc7frames2bit    -> your_design.bit
|-> openFPGALoader   -> the board, over USB
```

Simulation runs your design on your computer so you can test it without the board. Synthesis turns your SystemVerilog into the parts that exist on the chip, like lookup tables and flip-flops. Place and route decides where each part sits on the chip and checks that signals arrive within the 100 MHz clock. The next two tools come from a project called prjxray, and together they write the .bit file. The last tool sends that file to the Basys3 over the USB cable.

The whole setup takes 1.76 GB of disk. Most of it is nextpnr at 1.1 GB and the chip database at 344 MB. Vivado asks for 50 to 100 GB.

2. What do you need before you start?

You need three things that are probably half there already.

The terminal. Press Cmd and Space, type Terminal and press Enter. The window that opens is where every command in this guide goes. You paste a command, press Enter and wait until the prompt comes back before you paste the next one.

Apple's command line tools. This gives your Mac git, which downloads code from GitHub, and clang, which turns code into programs.

```
xcode-select --install
```

A window pops up and asks you to install. If it says the tools are already installed, you are done with this step.

Homebrew. This is an installer for developer tools. You type the name of a tool and it downloads a ready-made copy.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It asks for your Mac password. You will not see the letters while you type, which is normal. When it finishes it prints a few lines under the heading Next steps. Copy those lines, paste them into the terminal and press Enter. They tell your terminal where brew lives, and without them the next command answers command not found.

Check that it works.

```
brew --version
```

You should see a version number.

A code editor. You will create a few text files in step 8. I use VS Code, which you download from code.visualstudio.com. Do not use TextEdit for this. It saves rich text by default, and the tools cannot read that.

3. Which tools come ready-made?

Three of the five tools are on Homebrew, so you get them in one command. The same command also brings the helpers you will need in steps 5 and 7 to compile the other two.

```
brew install yosys openfpgaloader icarus-verilog \
    cmake ninja boost eigen pkg-config libomp python
```

The first three are the tools from the diagram. `cmake` and `ninja` are the programs that run a compile job. `boost`, `eigen`, `pkg-config` and `libomp` are pieces that nextpnr's code expects to find on your machine. `python` runs two of the steps in the chain.

This download is a few hundred MB, so give it some minutes.

Check that the three tools landed. The text after `#` is what I saw.

```
yosys -V           # Yosys 0.69+post
openFPGALoader --Version # v1.1.1
iverilog -V | head -1  # Icarus Verilog version 13.0 (stable)
```

Your version numbers can be newer. You only need each command to answer.

`openFPGALoader` already knows our board, and you can see that for yourself.

```
openFPGALoader --list-boards | grep basys3
# basys3    diligent    xc7a35tcpg236
```

`xc7a35tcpg236` is the exact name of the chip on the Basys3. You will see it again in step 6.

4. Why do you need a Python environment?

Two steps in the chain are Python scripts, and they need a few Python packages. Homebrew's Python refuses a plain `pip install` because it protects its own packages. The way around it is a `venv`, which is a private folder with its own Python where you are free to install things.

This also creates `~/fpga`, the folder that will hold everything from this guide. The `~` means your home folder.

```
mkdir -p ~/fpga
python3 -m venv ~/fpga/venv
~/fpga/venv/bin/pip install fasm pyyaml textx simplejson intervaltree
```

You will see a warning about the Antlr parser. It is normal. It means a slower Python parser gets used, and at lab sizes you will not feel it.

5. How do you build nextpnr-xilinx?

This is the place and route tool. Homebrew has no ready-made copy of it for macOS. What exists is its code on GitHub, so you download the code and your Mac turns it into a program. People call this compiling or building from source.

First you download it. `git clone` copies a GitHub project into a folder on your Mac. `--recursive` also brings the other projects that live inside it, and `--depth 1` skips the old history so the download is smaller.

```
cd ~/fpga
git clone --recursive --depth 1 https://github.com/openXC7/nextpnr-xilinx.git
cd nextpnr-xilinx
```

Then you compile it. The `cmake` command prepares the job and the `ninja` command does it.

```
cmake -B build -G Ninja \
  -DARCH=xilinx \
  -DCMAKE_BUILD_TYPE=Release \
  -DUSE_OPENMP=OFF \
  -DBUILD_GUI=OFF -DBUILD_PYTHON=OFF -DBUILD_TESTS=OFF

ninja -C build -j3
```

The `ninja` step takes about 10 minutes. You will see a counter like `[57/312]` going up, and your Mac may get warm.

Two parts of that command matter.

`-DUSE_OPENMP=OFF` is required. Apple's compiler has no OpenMP, and without this flag the build dies on the first file with `clang++: error: unsupported option '-fopenmp'`. OpenMP only makes one part of placement faster, and at CS223 design sizes you will not notice it missing.

`-j3` means three compile jobs run at the same time. On 8 GB of RAM a higher number makes the Mac run out of memory and slow down. With 16 GB or more you can raise it.

Check that it built.

```
~/fpga/nextpnr-xilinx/build/nextpnr-xilinx --version
# "nextpnr-xilinx" -- Next Generation Place and Route (Version 0.9.6)
```

6. How do you make the chip database?

`nextpnr` knows how to place and route, but it knows nothing about our chip. It needs a file that describes every part and every wire on the XC7A35T. That file does not come with the download, so you generate it once.

```
mkdir -p ~/fpga/chipdb
cd ~/fpga/nextpnr-xilinx

~/fpga/venv/bin/python xilinx/python/bbaexport.py \
  --xray xilinx/external/prjxray-db/artix7 \
  --metadata xilinx/external/nextpnr-xilinx-meta/artix7 \
  --device xc7a35tcbg236-1 \
  --constids xilinx/constids.inc \
  --bba ~/fpga/chipdb/xc7a35t.bba

build/bbasn -l ~/fpga/chipdb/xc7a35t.bba ~/fpga/chipdb/xc7a35t.bin
```

The first command takes about 35 seconds and writes a 256 MB text file. The second takes 3 seconds and packs it into an 89 MB binary file, which is the one `nextpnr` reads.

`--xray` and `--metadata` both point inside the folder you cloned in step 5. The raw chip data came along with `--recursive`, so there is nothing extra to download.

For a different board you would change `--device` to that board's chip name.

7. How do you build the prjxray tools?

nextpnr's result is a text file that lists every setting on the chip. The board wants a binary `.bit` file. prjxray has the two tools that do this translation. One is a Python script and the other one you compile, the same way as in step 5.

```
cd ~/fpga
git clone --recursive --depth 1 https://github.com/f4pga/prjxray.git
cd prjxray

~/fpga/venv/bin/pip install -e .

cmake -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_POLICY_VERSION_MINIMUM=3.5
cmake --build build --target xc7frames2bit -j3
```

If you open prjxray's own README, it starts by telling you to install Vivado 2017.2. Ignore that. It is for people who map a chip from zero, and our chip is already mapped.

`--recursive` is required here too. This project pulls in six smaller projects, and without them CMake stops with `Cannot specify include directories for target "yaml-cpp" which is not built by this project`.

`-DCMAKE_POLICY_VERSION_MINIMUM=3.5` is also required. The project is written for an older CMake, and a current CMake refuses it without this flag.

Check that the program is there.

```
ls ~/fpga/prjxray/build/tools/xc7frames2bit
```

If it prints the path back, the installation is finished. Everything after this is about your own project.

8. What goes in your project folder?

Your project needs its own folder. These two commands create one and move the terminal into it.

```
mkdir -p ~/cs223/blink
cd ~/cs223/blink
```

Then open VS Code, choose File and Open Folder, and pick `cs223/blink` inside your home folder. The folder needs five files.

The fastest way to get them is to download them. These two commands put all five files into the folder, with their tabs and indentation intact.

```
curl -L -o blink.zip https://noseydedrop.com/cs223/blink.zip
unzip blink.zip
```

If you would rather type them yourself, they are all printed below. Make a new file in VS Code for each one, give it the exact name shown, paste the content and save. Two things go wrong when you copy out of a PDF. A page number from the bottom of the page can land in the middle of a block, so look for a stray number after you paste. And the indented lines in the Makefile need real Tab characters, which section 8 covers at the end.

The example is a blinking LED, which is the smallest design that proves the whole chain works.

blink.sv is the design. LED 0 blinks once a second while switch 0 is on, and LED 15 follows switch 0 directly.

```

module blink (
    input logic clk,
    input logic [15:0] sw,
    output logic [15:0] led
);
    localparam int HALF_PERIOD = 50_000_000;

    logic [25:0] counter = '0;
    logic blink_state = 1'b0;

    always_ff @(posedge clk) begin
        if (counter == HALF_PERIOD - 1) begin
            counter <= '0;
            blink_state <= ~blink_state;
        end else begin
            counter <= counter + 1'b1;
        end
    end

    always_comb begin
        led = '0;
        led[0] = blink_state & sw[0];
        led[15] = sw[0];
    end
endmodule

```

blink_tb.sv is the testbench, the file that make sim runs. It flips switch 0 and checks that LED 15 follows.

```

`timescale 1ns/1ps

module blink_tb;
    logic clk = 0;
    logic [15:0] sw = '0;
    logic [15:0] led;

    blink dut (.clk(clk), .sw(sw), .led(led));

    always #5 clk = ~clk;

    initial begin
        $dumpfile("blink.vcd");
        $dumpvars(0, blink_tb);

        sw[0] = 0; #100;
        if (led[15] !== 1'b0) $error("sw[0]=0 should give led[15]=0");

        sw[0] = 1; #100;
        if (led[15] !== 1'b1) $error("sw[0]=1 should give led[15]=1");

        $display("TB passed");
        $finish;
    end
endmodule

```

blink.xdc is the pin file. It says which port of your design goes to which physical pin on the board. It is the same file you would use in Vivado, and you download it from Digilent.

```
curl -sL -o blink.xdc \  
https://raw.githubusercontent.com/Digilent/digilent-xdc/master/Basys-3-Master.xdc
```

Run that in the terminal and the file appears in your folder. Every pin line in it starts with #, which means it is switched off. Open the file in VS Code and remove the # from the lines your design uses. For `blink.sv` that means the two lines that end in `[get_ports clk]`, the sixteen lines with `sw[0]` to `sw[15]` and the sixteen lines with `led[0]` to `led[15]`. That is 34 lines and it is the whole edit. The `PACKAGE_PIN ... IOSTANDARD` syntax works in nextpnr as it is.

check_xdc.py compares your design's ports with the pin file before the build runs. You need it because nextpnr's own error for a pin mistake names the wrong port, and you end up searching in the wrong place.

```
#!/usr/bin/env python3  
import json, re, sys  
  
jf, xf = sys.argv[1], sys.argv[2]  
  
design = json.load(open(jf))  
top = None  
for name, mod in design["modules"].items():  
    if mod.get("attributes", {}).get("top"):  
        top = (name, mod); break  
if top is None:  
    name = list(design["modules"])[0]  
    top = (name, design["modules"][name])  
tname, tmod = top  
  
ports = set()  
for pn, info in tmod.get("ports", {}).items():  
    n = len(info.get("bits", []))  
    if n == 1:  
        ports.add(pn)  
    else:  
        ports.update(f"{pn}[{i}]" for i in range(n))  
  
xdc = set()  
for line in open(xf):  
    line = line.split("#")[0]  
    for m in re.finditer(  
        r"get_ports\s*\{?\s*([A-Za-z_]\w*(?:\[\d+\])?\)", line):  
        xdc.add(m.group(1))  
  
missing = sorted(ports - xdc)  
extra = sorted(xdc - ports)  
  
if missing:  
    print(f"No pin assignment in the xdc for ({tname}):")  
    for x in missing: print(f"    - {x}")  
if extra:  
    print("In the xdc but not in the design, probably a typo:")  
    for x in extra: print(f"    - {x}")  
  
if missing or extra:  
    sys.exit(1)  
print(f"xdc ok: {len(ports)} ports, all matched.")
```

Makefile is the file that connects the five tools. Each block in it is one step of the chain, and `make` runs them in the right order. The file name is exactly **Makefile**, with no extension.

```

TOP := blink
SRCS := blink.sv
XDC := blink.xdc

PART := xc7a35tcbg236-1
FPGA := $(HOME)/fpga
NEXTPNR:= $(FPGA)/nextpnr-xilinx/build/nextpnr-xilinx
CHIPDB := $(FPGA)/chipdb/xc7a35t.bin
XRAYDB := $(FPGA)/nextpnr-xilinx/xilinx/external/prjxray-db/artix7
PY := $(FPGA)/venv/bin/python
F2F := $(FPGA)/prjxray/utils/fasm2frames.py
F2B := $(FPGA)/prjxray/build/tools/xc7frames2bit
CHECKER:= ./check_xdc.py

.PHONY: all sim synth bit flash clean check

all: bit

check:
    @for t in yosys iverilog openFPGALoader; do \
        printf "%-16s " "$t; command -v $t || echo "MISSING"; done
    @printf "%-16s " nextpnr; [ -x $(NEXTPNR) ] && echo ok || echo MISSING
    @printf "%-16s " frames2bit; [ -x $(F2B) ] && echo ok || echo MISSING
    @printf "%-16s " chipdb; [ -f $(CHIPDB) ] && echo ok || echo MISSING
    @printf "%-16s " venv; [ -x $(PY) ] && echo ok || echo MISSING
    @printf "%-16s " fasm2frames; [ -f $(F2F) ] && echo ok || echo MISSING

sim: $(TOP)_sim
    ./${TOP}_sim

$(TOP)_sim: $(SRCS) $(TOP)_tb.sv
    iverilog -g2012 -o $@ $(SRCS) $(TOP)_tb.sv

synth: $(TOP).json
$(TOP).json: $(SRCS)
    yosys -p "synth_xilinx -flatten -abc9 -nobram -arch xc7 \
        -top $(TOP); write_json $@" $(SRCS)

$(TOP).fasm: $(TOP).json $(XDC)
    @python3 $(CHECKER) $(TOP).json $(XDC)
    $(NEXTPNR) --chipdb $(CHIPDB) --xdc $(XDC) --json $(TOP).json --seed 1 \
        --write $(TOP)_routed.json --fasm $@

$(TOP).frames: $(TOP).fasm
    $(PY) $(F2F) --part $(PART) --db-root $(XRAYDB) $< > $@

bit: $(TOP).bit
$(TOP).bit: $(TOP).frames
    $(F2B) --part_file $(XRAYDB)/$(PART)/part.yaml --part_name $(PART) \
        --frm_file $< --output_file $@
    @ls -lh $@

flash: $(TOP).bit
    openFPGALoader -b basys3 $(TOP).bit

clean:
    rm -f $(TOP)_sim $(TOP).vcd $(TOP).json $(TOP)_routed.json \
        $(TOP).fasm $(TOP).frames $(TOP).bit

```

There are two traps when you copy this. The first one is that the page number at the bottom of a

PDF page can land in the middle of the block, so check for a stray number after you paste. The second one is that the indented lines in a Makefile must start with a real Tab character. A PDF cannot hold tabs, and some web pages turn them into spaces. If you copied the file from a PDF, `make` will stop with `missing separator`. This command puts the tabs back.

```
perl -pi -e 's/^ +/\t/' Makefile
```

Run it once inside your project folder, and the error is gone.

`-g2012` tells Icarus to read the file as SystemVerilog. `--seed 1` makes builds repeatable, because without it nextpnr picks a random layout on every run.

9. How do you run it?

Plug the Basys3 into your Mac with the USB cable and switch it on. Then run these from inside your project folder.

```
make check      # is every tool in place
make sim        # simulation
make bit        # .sv -> .bit           (~4 seconds)
make flash      # load onto the board
```

`make check` prints one line per tool. Every line should end with a path or with `ok`. If one says `MISSING`, go back to the step that installs it.

`make sim` prints three lines, and the one you care about is `TB passed`.

```
VCD info: dumpfile blink.vcd opened for output.
TB passed
blink_tb.sv:23: $finish called at 200000 (1ps)
```

The last line only says the test reached its end. If a check fails you see a line with `ERROR` and the message from the testbench instead. The run also writes `blink.vcd`, which holds the value of every signal over time. A waveform viewer can open that file. I have not set one up yet, so this guide stops at the file.

`make bit` runs synthesis, the pin check, place and route and the two prjxray tools. A lot of text scrolls by. In the middle you should see `xdc ok: 33 ports, all matched.`, which is the pin check passing. If it stops there instead, it lists the port names you forgot to uncomment or mistyped. The whole thing takes about four seconds and ends by listing `blink.bit` with its size.

`make flash` sends the file to the board, and a good run ends like this.

```
Load SRAM: [=====] 100.00%
Done
ir: 1 isc_done 1 isc_ena 0 init 1 done 1
```

`done 1` means the chip accepted the design. Turn switch 0 on, and LED 15 lights up while LED 0 starts blinking. The design sits in SRAM, so it is gone when you switch the board off and you run `make flash` again.

10. How do you use it for your own lab?

Copy `Makefile` and `check_xdc.py` into your lab folder and change the first three lines of the `Makefile`.

```
TOP := traffic_light
SRCS := traffic_light.sv debounce.sv
XDC := lab4.xdc
```

TOP is the name of your top module. SRCS is every .sv file in the design, with a space between them. XDC is your pin file with the right lines uncommented.

`make sim` looks for a testbench named after TOP, so with the lines above it expects `traffic_light_tb.sv`.

The port names in your top module have to match the names in the pin file. Digilent's file calls them `clk`, `sw[0]`, `led[0]`, `seg[0]`, `an[0]`, `btnC` and so on. If your module says `clock` instead of `clk`, change the name inside `[get_ports ...]` on that line of the pin file. The pin check in `make bit` tells you which names do not match.

11. How do you set up VS Code?

You installed VS Code in step 2. Add the SystemVerilog extension for syntax colors and error marks.

```
code --install-extension mshr-h.veriloghdl
```

If the terminal says `code` is not found, open VS Code, press Cmd Shift P and run the command Shell Command: Install 'code' command in PATH.

To flash with one key, put this in `.vscode/tasks.json` inside your project.

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "FPGA: flash",
      "type": "shell",
      "command": "make flash",
      "options": { "cwd": "${fileDirname}" },
      "group": { "kind": "build", "isDefault": true },
      "presentation": { "reveal": "always", "clear": true },
      "problemMatcher": []
    }
  ]
}
```

Then bind a key in `keybindings.json`.

```
[
  { "key": "cmd+j",
    "command": "workbench.action.tasks.runTask",
    "args": "FPGA: flash" }
]
```

Now Cmd J builds the open design and loads it onto the board.

12. Which course file breaks?

One. The `SevSeg_4digit.sv` module that the course hands out has a broken port line, and you will hit it in the first lab that uses the seven-segment display.

```
output [6:0]seg, logic dp, // dp ends up with no direction
```

Whoever wrote it meant `output logic dp`, but the language does not read it that way. Vivado accepts the line anyway, and so does Icarus Verilog. Yosys stops with `ERROR: Module port 'dp' is neither input nor output`.

Write it like this and it synthesizes.

```
output [6:0] seg,  
output logic dp,
```

13. What was tested?

I used real CS223 code from old student repos on GitHub.

Module	What it exercises	Result
traffic_light_system.sv	FSM with <code>typedef enum</code> , <code>always_ff</code> , <code>always_comb</code>	18 cells
memory_module.sv	16x8 RAM inference, dual read port	4x RAM32M
debounce.sv	25-bit counter plus FSM	40 cells, CARRY4 chain
SevSeg_4digit.sv	Seven-segment multiplexing	46 cells, after the fix in step 12

All four in one design come out at 136 LUTs and 48 flip-flops. The timing check passes at 154.70 MHz against our 100 MHz clock, and the bitstream is ready in 4.5 seconds.

The SystemVerilog we use in labs went through with zero errors. That covers `parameter`, `generate`, `struct packed`, `$clog2`, `unique case`, packed arrays and `interface` with `modport`.

A few messy cases passed too. The file name can differ from the module name, the source file can carry Turkish characters and a BOM, and the folder path can have spaces in it.

14. What can it actually not do?

This chain is not a replacement for Vivado. It covers the small part of Vivado that CS223 labs use, and nothing more.

Ready-made blocks. Anything from Vivado's IP Catalog is out. That includes Block Design, IP Integrator, MicroBlaze and AXI, along with the Clocking Wizard and the BRAM, VGA and UART cores. I checked whether CS223 needs any of them. I searched 35 old student repos and 240 source files and found none. The course has you write clock dividers and memory by hand, which is the part this chain handles.

The graphical tools. Vivado shows you waveforms, a schematic of your design and a debugger that watches signals inside the running chip. This chain has none of those. Everything happens in the terminal and the results are text.

Vivado project files. This chain does not create a Vivado project. Your `.sv` and `.xdc` files are the same ones Vivado uses, so you can still open them in Vivado on a lab computer.

BRAM. The Makefile passes `-nobram`, so arrays become distributed RAM. That is fine at lab sizes.

Large multipliers. I have not tested a design with a big multiplication. Yosys may map it onto the chip's DSP blocks, and I do not know if the rest of the chain handles those.

Other boards. I only tested the Basys3. Another 7-series chip needs its own chip database, which is the command in step 6 with a different `--device`.

15. What should you be concerned about?

Two tools read your code, and they do not agree on everything. Icarus Verilog reads it for simulation and Yosys reads it for synthesis. Each one supports its own part of SystemVerilog. Code that passes `make sim` can still fail in `make bit`, and step 12 is a real example. The features I tested are listed in step 13. Anything more advanced than that list is untested, and Yosys may reject it. When it does, the error usually names the file and the line. Rewriting that line in a simpler way has fixed it for me.

If your lab is checked in Vivado, try it in Vivado too. Your `.sv` and `.xdc` files are the same ones Vivado reads, so I expect a design that works here to work there. I have not compared the two lab by lab. I would not make the lab demo the first time you open your design in Vivado.

If your report needs a waveform picture, this guide does not get you there yet. `make sim` tells you pass or fail and writes a `.vcd` file. Opening that file as a waveform needs a viewer, and I have not set one up.

These tools are unofficial. AMD does not make or support them. `nextpnr-xilinx` calls itself an experiment. `prjxray` worked out the bitstream format on its own, without AMD's documentation. They worked for every lab I tried. If something behaves strangely on the board, build the same design in Vivado before you blame your code.

The timing number is an estimate. The 154.70 MHz in step 13 comes from `nextpnr`'s own model of the chip. Vivado's timing report is the official one. At 100 MHz with lab-sized designs the margin is large, so I did not worry about it.

The code you download keeps changing. `git clone` brings the newest version of each project, which may differ from what I used. The versions I used are in step 17. If a build step that worked for me fails for you, this is the first thing to suspect.

I tested on one machine. It was an M2 with 8 GB of RAM and macOS 15. I have not tried an Intel Mac, a different macOS version or a different board.

16. What went wrong during my setup?

These cost me time, so you can skip them.

oss-cad-suite has no Xilinx support. It is YosysHQ's ready-made bundle with a macOS build, and it looks like the obvious first try. It ships `nextpnr` for `ice40`, `ecp5`, `nexus` and Gowin chips. I downloaded 497 MB before I found out Xilinx was missing.

Missing submodules show up one at a time. `prjxray` failed twice before I added `--recursive`, first on `yaml-cpp` and then on `googletest`. Nothing gives you the full list up front.

Builds differ from run to run by default. `nextpnr` picks a random seed. With `--seed 1` the `.fasm` and `.frames` files come out identical byte for byte. The `.bit` file still differs in one place, a timestamp in the header at byte 95. That is a standard Xilinx field and it does not change the circuit.

nextpnr's pin errors name the wrong port. Type `led` as `ledd` in the pin file and it says `ERROR: port led[0] of type PAD has no IOSTANDARD property`. That is a different port than the broken one. `check_xdc.py` runs before `nextpnr` and names the real mistake.

17. Which versions did I use?

Yosys	0.69+post (git 143eb14f)
nextpnr-xilinx	0.9.6 (openXC7)
openFPGALoader	1.1.1
Icarus Verilog	13.0 (stable)

prjxray-db	0.9.1-30-g1768fb3 (artix7)
Board	Digilent Basys3
Chip	Artix-7 XC7A35T-1CPG236C
Machine	Apple M2, 8 GB RAM, macOS 15 (Darwin 24.2.0)

18. What is not verified?

- The combined four-module test went as far as a bitstream. I did not load it onto the board because the board was not with me at the time. The designs I loaded and confirmed on hardware are `blink`, a switches-to-LEDs design and a Lab 2 adder/subtractor set.
- The tools depend on Homebrew's `PATH`, so they will not run in a stripped environment.
- I have not opened a waveform yet. `make sim` writes the `.vcd` file and the guide stops there.